

Quine: A POSIX-Native Runtime for Large Language Model Agents

Hao Ke
Independent Researcher, Bloomington, IN, USA
i@kehao.me

March 2026

Abstract

I present Quine, a runtime architecture that realizes LLM agents as native POSIX processes. The mapping is explicit: identity is PID, interface is standard streams and exit status, state is memory, environment, and filesystem, and lifecycle is `fork/exec/exit`. A single executable implements this model by recursively spawning fresh instances of itself. This placement makes process isolation, composition, and resource control available through existing OS mechanisms rather than reimplementing them in an application-layer framework. I discuss the resulting trade-offs, including error amplification in pipelines, loss of latent state at process boundaries, and prefill overhead from stateless execution, and identify several limitations of the POSIX model that motivate future extensions. A reference implementation of Quine is publicly available.

Introduction

“Write programs that do one thing and do it well. Write programs to work together. Write programs to handle text streams, because that is a universal interface.” — Doug McIlroy (McIlroy 1978)

The Problem

The rapid development of Large Language Model (LLM) agents has led to a proliferation of frameworks designed to manage their lifecycle, memory, and communication. Systems such as LangChain (LangChain, Inc. 2023), AutoGen (Wu et al. 2023), and CrewAI (CrewAI, Inc. 2024) have successfully democratized agent development by providing high-level abstractions for these tasks. However, a structural pattern has emerged: these systems primarily expose agent abstractions at the application layer, even when the underlying execution still relies on OS services.

This pattern introduces complexity that compounds as systems scale. By managing agents as objects within a single application process, frameworks frequently reconstruct mechanisms the operating system already provides:

- **Fault isolation** is simulated through exception handlers and try-catch blocks, rather than enforced by hardware-backed address space separation.

- **Context switching** between agents requires application-level scheduling logic, rather than delegating to the kernel’s mature scheduler.
- **Message passing** is mediated by in-process queues or database-backed channels, rather than kernel-managed pipes with backpressure.

The operating system, having evolved over five decades to solve these problems for classical software, is less commonly treated as a first-class runtime substrate for agents.

My Approach

Recent progress in tool calling and structured output makes it practical to bind natural-language reasoning to a small set of system-level operations. Rather than building another application-layer framework, Quine treats the operating system as the agent runtime. It consists of two components:

Component 1: A Protocol. A disciplined mapping from agent concepts to POSIX primitives across four dimensions:

- **Identity** — The agent’s unique identifier is its process ID (PID), assigned by the kernel.
- **Interface** — The agent communicates through standard streams (`stdin/stdout/stderr`) and reports outcomes via exit status.
- **State** — The agent’s memory is process memory (cleared on exit), environment variables (inherited by children), and filesystem (persistent).
- **Lifecycle** — The agent spawns children via `fork/exec`, continues itself via `exec`, and terminates via `exit`.

Section 2 elaborates each dimension; Figure 1 provides the complete mapping.

Component 2: A Single-Image Runtime. A unitary executable that implements this protocol. When an agent spawns a child, it instantiates the same runtime image with different arguments. Parent and child share code but diverge in state. This recursive structure means the runtime and the agent template are the same artifact—there is no separate “agent definition” language or configuration format.

Contributions

This paper makes four contributions:

1. **A disciplined agent-to-POSIX mapping** across identity, interface, state, and lifecycle (Section 2).
2. **A unitary recursive runtime** that realizes this mapping through self-instantiation—a single binary that spawns instances of itself (Section 3).
3. **A runtime design** that preserves OS-level separation between mission (`argv`) and material (`stdin`) during prompt construction (Section 4.2).
4. **Native process-level composition and isolation** through standard streams, exit status, and address-space boundaries (Section 4.1, 4.3).

Scope and Non-Claims

This paper presents Quine as a protocol and implementation, not as a paradigm shift or a replacement for existing frameworks. I make no claims about superior end-task performance compared to application-layer approaches; the contribution is architectural. The POSIX mapping represents Layer 1 of a broader vision; Layers 2–5 (Section 6) are future work and are included only to clarify the limits of what POSIX alone can provide.

The POSIX Mapping

This section expands each dimension of the protocol defined in Section 1, using POSIX primitives (IEEE 2017). The mapping adopts a disciplined one-to-one correspondence: each agent concept maps to a primary POSIX primitive, and the semantics are inherited directly from the operating system.

Identity

Agent Concept	POSIX Primitive	Semantics
Agent instance	Process (PID)	Kernel-managed identity
Agent boundary	Address space	Memory isolation between agents

An agent does not need a framework-assigned UUID. The kernel provides a unique identifier—the PID—which is globally unique within the system for the process lifetime. This identifier is used by the scheduler, the memory manager, and the signal subsystem; Quine inherits these associations rather than reimplementing them.

The address space boundary defines what memory an agent can access. Two agents (two processes) cannot read or write each other’s memory without explicit arrangement (shared memory segments, files). This isolation is enforced by the MMU at hardware level, not by convention or access control lists in application code.

Interface

Agent Concept	POSIX Primitive	Semantics
Initialization	<code>argv</code>	Immutable mission description
Perception	<code>stdin</code>	Data input / material
Deliverable	<code>stdout</code>	Data output / deliverable
Diagnostics	<code>stderr</code>	Diagnostics channel
Outcome	<code>exit status</code>	Success (0) or failure (>0)

The `argv` array is set at process creation and cannot be modified during execution. This immutability makes it suitable for carrying the agent’s mission—the high-level instruction that defines what the agent should accomplish. Because `argv` is separate from `stdin`, the instruction channel is structurally distinct from the data channel.

Standard input (`stdin`) carries the material the agent operates on. Standard output (`stdout`) carries the agent’s deliverable—its contribution to downstream processes. Standard error (`stderr`) carries diagnostics: progress indicators, warnings, and error messages that should not pollute the deliverable stream.

The exit status is a small integer (0–255) that signals the outcome of execution. By convention, 0 indicates success; non-zero values indicate various failure modes. This signal is consumed by the parent process via `wait()`, enabling conditional execution (`&&`, `||`) in shell scripts.

State

Agent Concept	POSIX Primitive	Semantics
Ephemeral	Process memory	Cleared on <code>exec</code> / <code>exit</code>
Scoped	Environment variables	Preserved across <code>exec</code> ; inherited by children
Global	Filesystem	Persistent; shared (default) or isolated via namespaces

This three-tier model maps directly to process lifetime scopes:

Ephemeral state (process memory) exists only while the process runs. When the process calls `exec` or terminates, this state is lost. For LLM agents, this corresponds to the “working memory” accumulated during a single execution—the context window contents, intermediate computations, and any in-memory data structures.

Scoped state (environment variables) survives `exec` and is copied to child processes. This makes environment variables suitable for passing compact metadata between generations: progress markers, configuration flags, or compressed summaries of prior execution. The copy-on-fork semantics mean children can modify their environment without affecting the parent.

Global state (filesystem) persists beyond process lifetime and is visible to all processes (subject to permissions). This is the only state tier that survives both `exec` and process termination. For agents, the filesystem serves as long-term memory, shared artifacts, and coordination medium.

Lifecycle

Agent Concept	POSIX Primitive	Semantics
Spawn	<code>fork + exec</code>	Create child process
Continue	<code>exec (self)</code>	Replace current process, preserve identity
Terminate	<code>exit(status)</code>	Signal outcome to parent

An agent can **spawn** children to delegate work. The parent calls **fork** to create a copy of itself, then the child calls **exec** to replace its image with a fresh Quine instance (possibly with different **argv**). The parent can wait for the child to complete, or continue executing while monitoring child status asynchronously.

An agent can **continue** itself by calling **exec** with its own image. This replaces the process image—clearing process memory—while preserving PID and environment variables. For LLM agents, this provides a mechanism to escape context limits: the agent can checkpoint progress in environment variables, then **exec** to start fresh with a clean context window.

An agent **terminates** by calling **exit** with a status code. This releases all resources (memory, file descriptors, child processes if the parent does not wait), notifies the parent, and makes the exit status available for inspection. The three operations—spawn, continue, terminate—combined with the state hierarchy, define the lifecycle model.

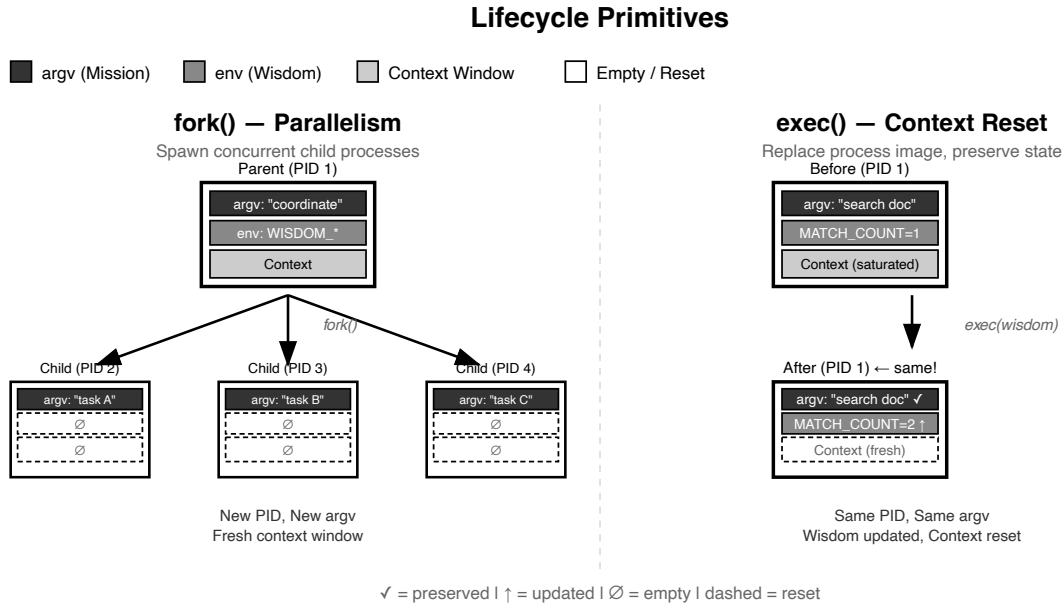


Figure 1: Agent Lifecycle: The three lifecycle operations (spawn, continue, terminate) and their relationship to state tiers. Spawn creates children via fork/exec; continue replaces the process image while preserving identity; terminate signals completion to the parent.

The Runtime

This section describes the implementation—a unitary executable that serves as both runtime and agent template.

Host-Guest Architecture: Separation of Control and Compute

Quine enforces a physical separation between deterministic control flow and probabilistic computation. This separation is not merely architectural preference; it reflects a fundamental asymmetry

in where state lives and how failures manifest.

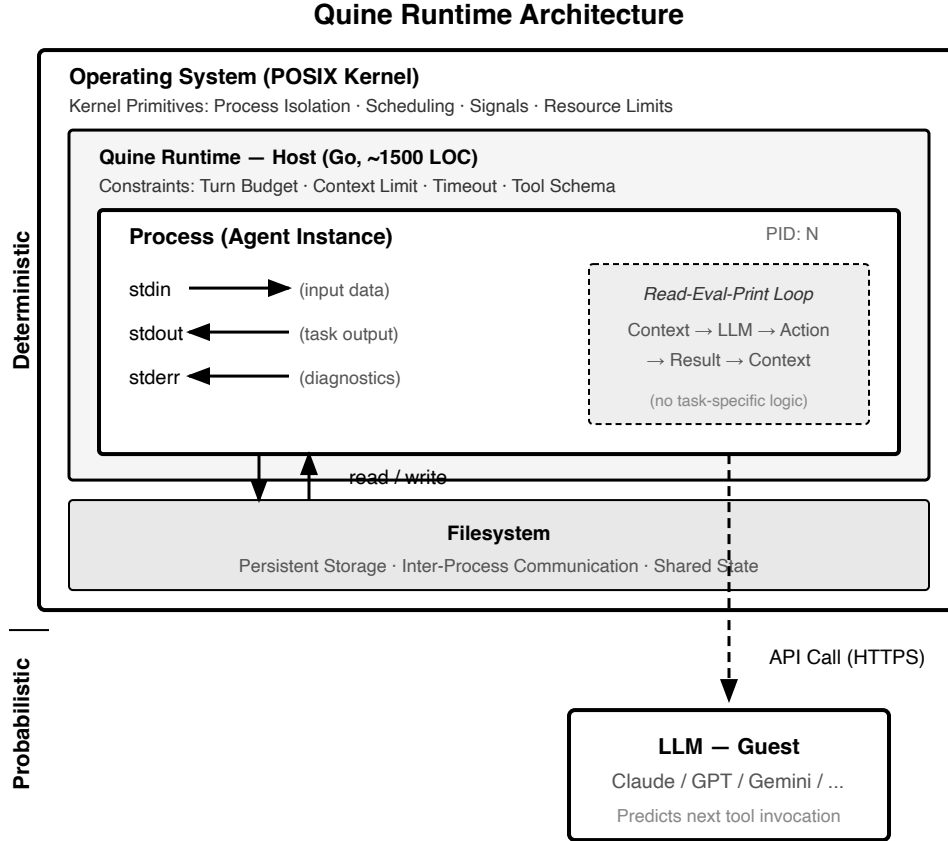


Figure 2: Host-Guest Architecture: The runtime separates deterministic control (Host) from probabilistic computation (Guest). The Host manages syscalls, I/O, and signals; the Guest provides decisions, tool calls, and reasoning.

Host (Local OS Process). The Host is a conventional compiled program that maintains the agent’s context, lifecycle, and filesystem state. It is responsible for:

- Parsing `argv` to extract the mission
- Reading `stdin` to obtain material
- Managing file descriptors and child processes
- Serializing state for transmission to the Guest
- Executing tool calls returned by the Guest
- Writing deliverables to `stdout` and diagnostics to `stderr`
- Calling `exit` with an appropriate status code

Because cognition is delegated to remote endpoints, the local process footprint remains small. Memory is primarily used for I/O buffers, the serialized conversation history, and control state. This keeps the local runtime lightweight and allows agent instances to be managed as ordinary OS processes—schedulable, killable, and resource-bounded by standard tools.

Guest (Remote LLM API). The Guest acts as a stateless cognitive oracle. It receives the

serialized POSIX state (mapped to prompts) and returns structured tool invocations. The Guest has no persistent state between calls; all context must be provided in each request. This statelessness is a design choice: it ensures that the Host maintains authoritative state and that the Guest can be replaced, rate-limited, or load-balanced without coordination.

The binary contains no task-specific logic. Task-specific reasoning—what shell commands to run, when to spawn children, how to interpret results—is delegated to the Guest. The Host is purely reactive: it executes what the Guest requests, captures the results, and asks again.

Context Assembly. The Host avoids application-layer restructuring of the data stream and instead preserves OS-level I/O boundaries when assembling context. Specifically:

1. The mission (**argv**) is placed in the System Prompt, establishing the instruction context.
2. Material (**stdin**) enters the User Message, clearly marked as input data.
3. Tool outputs (shell command results, child process outputs) are returned as Tool Result messages.

This layered structure aligns with instruction hierarchy training (Wallace et al. 2024), where models learn to prioritize System > User > Tool. The Host does not rewrite or summarize the data stream; it annotates boundaries and lets the Guest operate on the structured context.

Tool Interface. The Host exposes four tools to the Guest—**sh**, **fork**, **exec**, **exit**—named after and semantically aligned with their POSIX counterparts:

Tool	POSIX Analog	Semantics
sh	system()	Execute shell command, return stdout/stderr/exit status
fork	fork + exec	Spawn child process(es) running ./quine ; block until completion; return outputs
exec	exec	Replace current process image with fresh ./quine ; preserve env vars
exit	exit()	Terminate with status code

The **sh** tool is the primary interaction mechanism. When the Guest wants to read a file, list a directory, run a compiler, or perform any system operation, it issues a shell command. The Host executes the command in a subprocess, captures stdout and stderr, and returns them to the Guest as a tool result.

The **fork** tool spawns one or more child Quine processes. Each child receives its own **argv** (the sub-mission) and optionally **stdin** (sub-material). The parent blocks until all children complete, then receives their stdout, stderr, and exit status as a structured result. This enables delegation without leaving the process model.

The **exec** tool replaces the current process image. The Guest can specify new **argv** and pass data through environment variables. This is how agents manage context growth: checkpoint progress in env vars, then **exec** to continue with a fresh context window.

The `exit` tool terminates the process. The Guest specifies a status code and optionally a final message. The Host writes any pending output, then calls `exit()`.

POSIX Conformance: The I/O Contract

Quine enforces a **Quad-Channel Protocol** that decouples intent, data, execution, and observability:

Channel	Stream	Content	Consumer
Intent	<code>argv</code>	Mission / instructions (text)	Runtime → System Prompt
Material	<code>stdin</code>	Input data (bytes)	Runtime → User Message
Deliverable	<code>stdout</code>	Pure output (bytes)	Agent → Downstream
Signal	<code>stderr</code>	Diagnostics / failures	Agent → Supervisor
Outcome	<code>exit status</code>	Success (0) / failure (>0)	Process → Parent <code>wait()</code>

External view. From the shell’s perspective, Quine is a standard POSIX filter. Standard input flows in, standard output flows out, standard error carries diagnostics, and exit status signals success or failure. This means Quine can be used anywhere a traditional Unix filter can be used: in pipelines, in shell scripts, as a subprocess of other programs.

Internal mechanism. To maintain clean context windows while honoring this contract, the runtime uses file descriptor passing. When the agent spawns shell commands, the runtime passes additional file descriptors for the agent to write deliverables directly to the runtime’s `stdout`. This separation allows the agent to:

1. Capture command output for reasoning (returned as tool results)
2. Emit deliverables to downstream processes (written to runtime’s `stdout`)

Without this separation, the agent would face a dilemma: either pollute the deliverable stream with intermediate command outputs, or lose visibility into command results. The file descriptor mechanism resolves this by providing separate channels for cognitive feedback and external output.

Illustrative Shell Compositions

Quine’s adherence to standard streams enables composition with classic Unix utilities and multi-agent pipelines.

Example 1: Single-process replacement. A cognitive filter that understands intent:

```
cat server.log | ./quine "Extract lines indicating auth failures" | sort | uniq -c
```

Here Quine replaces `grep` in a traditional pipeline. The cognitive filter receives log lines on `stdin`, applies semantic understanding to identify authentication failures, and emits matching lines to `stdout`. Downstream tools (`sort`, `uniq -c`) process the output normally.

Example 2: Implicit DAG. Three agents form a reasoning chain:

```
git diff HEAD~1 | \
  ./quine "Identify the changed components" | \
  ./quine "Assess risk level for each change" | \
  ./quine "Generate a review checklist"
```

Each agent receives the previous agent’s output as stdin and contributes its analysis to stdout. The shell provides the topology; no orchestrator or shared memory is required. Each agent runs in a separate process with separate context.

Example 3: Control flow without framework. Exit codes drive branching:

```
./quine "Is this a security vulnerability? Exit 0=yes, 1=no" < report.md && \
  ./quine "Draft disclosure timeline and patch plan" < report.md
```

The first agent classifies the input and signals its determination via exit status. The shell’s `&&` operator conditionally executes the second agent only if classification returns “yes.” This is cognitive branching using standard shell control flow.

Implementation Scale

The runtime is implemented in approximately 7,700 lines of Go (including LLM provider abstractions and error handling), compiled to a single ~9.8 MB binary. The runtime contains no domain-specific task logic; decision-making is delegated to the Guest (LLM).

The implementation supports multiple LLM providers (Anthropic Claude, OpenAI GPT, Google Gemini) through a provider abstraction layer. Provider selection is controlled by environment variables, enabling the same binary to use different backends without recompilation.

Source code is available at <https://github.com/kehao95/quine>.

Properties

These properties are not implemented features; they are structural consequences of mapping agents to POSIX processes.

By representing agents as processes, Quine inherits mechanisms refined over five decades of Unix evolution: the security model (capabilities, namespaces, seccomp), the resource model (cgroups, ulimit, OOM killer), and the concurrency model (fork/wait, signals, job control). I highlight three categories most relevant to LLM agents; the full Unix toolkit remains available.

Structural Composition

Recursive Composition. The architecture exhibits structural self-similarity: a child process is isomorphic to its parent. Both run the same binary and expose the same interface (`argv/stdin/stdout/exit`), regardless of depth in the process tree.

This property removes the distinction between “worker” and “manager” nodes—every node can spawn and coordinate its own subtree. A child delegated to “summarize this document” can itself spawn children to “extract key points from each section.” From the parent’s perspective, a

delegated subtree presents a single process-level interface: `stdout`, `stderr`, and exit status. The internal structure of the subtree is encapsulated.

Mechanism: When a parent calls `fork`, the child inherits the same binary image. The child’s `argv` (passed via `exec`) determines its mission. The child’s exit status reports success or failure. This is standard Unix process semantics; Quine does not add any coordination logic.

Implication: Complex workflows can be expressed as process trees without requiring a separate workflow definition language. The shell itself becomes the orchestrator.

Universal Composability. Because Quine is a standard POSIX executable (Section 3.2), it composes with any Unix tool—shell scripts, `xargs`, GNU `parallel`, `cron`, or another Quine instance. An agent can process the output of `grep`, feed its output to `awk`, or pipe to another agent.

Mechanism: Quine reads from file descriptor 0 (`stdin`), writes to file descriptor 1 (`stdout`), emits diagnostics to file descriptor 2 (`stderr`), and reports outcome via exit status. These are the same conventions followed by `cat`, `grep`, `sed`, and every other Unix filter.

Implication: Existing Unix infrastructure—shell scripts, makefiles, cron jobs, CI pipelines—can incorporate cognitive agents without modification. There is no new protocol to learn, no daemon to run, no message format to serialize.

Execution Safety

Fault Isolation. Each agent occupies a separate address space. The kernel enforces memory protection: a child crash (SEGV, OOM, infinite loop) cannot corrupt the parent’s memory.

Mechanism: The Memory Management Unit (MMU) enforces address space separation at hardware level. When a process attempts to access memory outside its address space, the MMU generates a fault. The kernel handles this fault by terminating the offending process and notifying its parent via `wait()`.

Implication: A buggy or malicious child cannot corrupt the parent’s state. Upon child failure, the parent receives the exit status and any `stderr` output, then decides how to proceed: retry, skip, abort, or escalate. This isolation comes from the hardware, not from defensive programming in the framework.

Instruction-Data Separation. The architecture separates mission from material at the OS level: `argv` carries the mission (instruction), while `stdin` carries material (data). These channels are structurally distinct—data from `stdin` cannot overwrite the mission in `argv`.

Mechanism: In POSIX, `argv` is set at process creation (via `exec`) and cannot be modified during execution. Standard input is a file descriptor that carries data during execution. The Host maps these to separate locations in the LLM context: `argv` → System Prompt, `stdin` → User Message. This mapping preserves the structural separation through the cognitive layer.

Implication: Combined with instruction hierarchy training (Wallace et al. 2024) (System > User > Tool), this provides a structural basis that may reduce certain classes of prompt manipulation risks. An attacker who controls the data stream (`stdin`) cannot directly inject instructions into the mission channel (`argv`) or the System Prompt. However, systematic security evaluation is outside the scope of this paper; I note this as a structural property, not a security claim.

Resource Limits. Standard Unix mechanisms (`ulimit`, `timeout`, `cgroups`) bound execution to finite budgets. A child process can be constrained to maximum CPU time, memory, file descriptors, and other resources.

Mechanism: The parent can set resource limits before spawning a child (via `setrlimit`) or enforce them externally (via `cgroups`, `timeout` wrapper). When a limit is exceeded, the kernel terminates the process and reports the cause.

Implication: Runaway agents—infinite loops, memory leaks, fork bombs—are bounded by kernel enforcement, not by hoping the agent respects soft limits.

Execution Continuity

Continuation across `exec`. The `exec` syscall replaces the current process image while preserving PID and environment variables. This enables an agent to clear accumulated context (and potential hallucinations) while retaining compressed state in `env` vars.

Mechanism: When a process calls `exec`, the kernel replaces the process image (code, heap, stack) but preserves:

- Process ID (PID)
- Parent process ID (PPID)
- Environment variables
- File descriptors (unless marked close-on-exec)
- Signal handlers (reset to defaults)

Process memory—including the LLM context window accumulated during execution—is cleared.

Implication: An agent approaching context limits can checkpoint its progress in environment variables (e.g., `PROGRESS=step3`, `SUMMARY="completed phases 1-2"`), then `exec` to continue with a fresh context. The new instance can read these variables and reconstruct relevant state without inheriting the full history.

This reframes part of the memory problem as deterministic reconstruction. Rather than carrying forward an ever-growing context, the agent maintains a compressed representation in `env` vars and reconstructs working context as needed. The raw history can persist on the filesystem for reference; the cognitive context is assembled selectively.

Feedback via `stderr` and exit status. The `stderr` stream and exit status form a feedback channel from child to parent. A failing child may emit diagnostic information on `stderr` before terminating; the parent reads this diagnostic and adjusts subsequent spawning strategies accordingly.

Mechanism: Standard error is a separate file descriptor (fd 2) that carries diagnostic output. The parent captures this stream when spawning children. Exit status is a 0-255 integer set by `exit()` and retrieved by `wait()`.

Implication: Parents can implement retry logic, fallback strategies, or error aggregation based on child feedback. This is not unique to Quine—it is standard Unix process supervision—but it applies directly to agent coordination.

Demonstration Evidence

The following observations demonstrate that the POSIX-native interface is operational in practice. They are included to show operational feasibility, not to claim state-of-the-art performance. These observations demonstrate operational feasibility rather than competitive benchmark performance.

Mission-First Streaming (MRCR Evaluation)

Task. I evaluated Quine on an MRCR-style needle retrieval task (OpenAI 2024) spanning 4K–279K tokens across 8 samples. The task requires finding the N-th occurrence of a specific content type (e.g., “the 2nd poem about tapirs”) in a long multi-turn conversation. This is a challenging retrieval task because the agent must:

1. Process a large document that may exceed context limits
2. Track ordinal position (first, second, third occurrence)
3. Return the exact content, not a summary

Setup. Both Quine and baseline used the same model family (Claude Sonnet). The baseline represents a direct full-context API call: the entire conversation is loaded into context, and the model is asked to find the target. Quine receives the same conversation on stdin but processes it incrementally, with the mission (“find 2nd poem about tapirs”) specified in `argv` before streaming begins.

Metric. Accuracy is measured by exact-match scoring against the ground-truth target. A score of 1.0 indicates exact match; partial matches receive scores between 0 and 1 based on content overlap. The \$ \$0.996 notation indicates scores very close to 1.0, typically differing only in whitespace or minor formatting.

Procedure. For each of the 8 samples:

1. Baseline: Load full conversation into context, query for target
2. Quine: Stream conversation to stdin, mission in `argv`, let agent process incrementally

Results.

Context Length	Quine Accuracy	Baseline Accuracy	Notes
4K–7K (5 samples)	\$ \$0.996 (all)	Failed 2/5	Baseline failures due to retrieval errors
178K–279K (3 samples)	1.000 (all)	Failed 3/3	Baseline exceeded context limits

Interpretation. The baseline (direct API call with full conversation loaded) failed on 5 out of 8 samples: 2 failures in short contexts (retrieval errors) and 3 failures in long contexts (exceeded context limits or timeout). Quine achieved \$ \$0.996 on all samples.

For long contexts, agents used `exec` to manage context growth. One run required 9 `exec` cycles to process a 178K-token sample: the agent would process a portion of the stream, checkpoint progress in environment variables, then `exec` to continue with a fresh context. This illustrates that `argv/stdin` separation, streaming, and `exec` continuation can work together to handle tasks that exceed single-context limits.

Limitations of this evaluation. Sample size is limited (N=8). The baseline represents one reasonable approach (full-context loading), not necessarily the optimal application-layer strategy. Results are illustrative of feasibility, not a benchmark claim.

Shell-Native Composition

In exploratory runs (not formal evaluation), agents successfully used shell job control for parallel coordination. Agents adopted `&` for background workers, `wait` for synchronization, and Unix text tools (`sort -u`) for reduction. This illustrates framework-free orchestration through POSIX composition, though I have not quantified the frequency or reliability of this behavior.

Continuation across `exec`

In exploratory runs, agents used repeated `exec` continuation to refresh context while carrying forward compact execution state in environment variables. The pattern typically involved:

1. Processing input until context grew large
2. Summarizing progress in env vars (e.g., `FOUND_COUNT=3`)
3. Calling `exec` to continue with fresh context
4. Reading env vars to resume from checkpoint

This behavior emerged without explicit instruction, suggesting the model understood the semantics of `exec` from the tool description. However, systematic evaluation of when and how reliably this emergence occurs is future work.

Reproducibility

The implementation, prompts, execution logs, and scripts used for the reported runs are available in the public repository at <https://github.com/kehao95/quine>. Full execution logs (JSONL format) and supporting artifacts for the MRCR evaluation are maintained in the repository.

Related Work and Limitations

Related Systems

I classify current agent systems by their structural relationship to the operating system. This classification focuses on runtime structure rather than end-task capability. This taxonomy considers where the agent’s identity, lifecycle, and isolation are managed—not the features they provide to developers.

Host-Coupled Extensions

Systems such as Cursor and GitHub Copilot are integrated within host environments (IDEs) and lack independent process lifecycles. The agent exists as a component of the host application; it cannot be spawned, killed, or composed independently. This tight coupling enables deep integration (inline suggestions, context from editor state) but limits composition with external tools.

Structural distinction: The agent is not addressable by PID; it has no independent stdin/stdout; termination is tied to the host application.

Application-Layer Schedulers

Frameworks such as LangGraph, AutoGen (Wu et al. 2023), and CrewAI (CrewAI, Inc. 2024) provide user-space scheduling and message buses. Agents are implemented as in-process objects (Python classes, coroutines) with simulated fault isolation through exception handling.

These systems have successfully demonstrated multi-agent coordination and have large ecosystems of tools and integrations. Their design optimizes for developer productivity and rapid prototyping.

Structural distinction: Agents share a single OS process. A crash in one agent (unhandled exception, memory corruption) can affect others. “Isolation” is implemented through language-level mechanisms (separate objects, try-catch), not hardware-enforced address space separation. Scheduling is managed by the framework, not the kernel.

Sandboxed Loops

Systems such as Devin, OpenHands (Wang et al. 2024), and SWE-agent (Yang et al. 2024) use an external controller to manage containerized tools. The agent’s “body” (shells, browsers, file access) runs in an isolated sandbox; the “brain” (LLM) runs in the controller process.

This architecture provides strong tool isolation—a runaway shell command cannot escape the container—while centralizing cognitive coordination. It has proven effective for complex software engineering tasks.

Structural distinction: The sandbox isolates tools, not the agent itself. The cognitive loop (prompt → LLM → action → observation) runs in a single controller process. Multiple “agents” in such systems are typically coroutines or threads within this controller, not separate OS processes.

Pseudo-OS Middleware

Systems such as AIOS (Mei et al. 2024), agentOS (Li et al. 2026), and UFO 2 (Zhang et al. 2025) implement OS-like abstractions in user space. They provide schedulers, memory managers, and IPC mechanisms for agents, using terminology borrowed from operating systems.

These systems recognize that agent management resembles process management and attempt to provide similar abstractions. However, the abstractions are simulated at the application layer rather than mapped to actual kernel primitives.

Structural distinction: Despite OS-inspired naming, agents in these systems are typically objects or threads within a single host process. “Process isolation” is simulated through software boundaries,

not hardware-enforced address spaces. The “scheduler” is a user-space dispatcher, not the kernel’s CFS. Resources (memory, CPU) are accounted at the framework level, not by cgroups or ulimit.

Quine’s Position

Quine represents a distinct design point. To my knowledge, existing agent runtimes primarily manage agents at the application layer or use the OS as a tool sandbox, rather than directly realizing each agent as a native POSIX process with the full reasoning-and-acting loop exposed through standard process interfaces.

This is a structural distinction rather than a claim of superior end-task performance. Application-layer frameworks offer flexibility, rich ecosystems, and rapid development cycles. Quine trades some of this flexibility for structural properties inherited from the OS.

The key differences:

Aspect	Application-Layer	Quine
Scheduler	Framework dispatcher	OS scheduler (CFS)
Message format	Framework-specific	Standard streams (stdin/stdout)
Lifecycle	Object instantiation	<code>fork/exec/exit</code>
Isolation	Exception handling	Process address spaces
Resource control	Framework accounting	cgroups, ulimit

Architectural Limitations of POSIX

POSIX was designed to manage execution, not cognition. While it addresses the external lifecycle of an agent—identity, communication, termination—the internal cognitive state remains opaque. The file descriptor defines the extent of its abstraction.

I identify four structural mismatches that necessitate extensions beyond the POSIX model presented here:

Opaque Internal State (The Structure Problem)

Process memory is a private black box. An agent’s “mind”—context window contents, attention patterns, intermediate reasoning—is inaccessible to the OS. Debugging requires parsing side-effects (logs, outputs) rather than inspecting state directly.

Manifestation: When an agent produces unexpected output, diagnosis involves reading logs and inferring internal state. There is no equivalent of `/proc/[pid]/maps` for cognitive state.

Direction: Addressed by Layer 2 in Section 6.

Global Namespace Pollution (The Space Problem)

POSIX enforces a single global file hierarchy. A process sees the entire filesystem (subject to permissions), whether relevant or not. For agents, this “shared reality” creates two problems: irrelevant files become distractions, and global visibility violates the principle of least privilege.

Manifestation: An agent tasked with editing `src/main.go` can also access `~/.ssh/id_rsa`. File-level permissions help but do not provide per-task scoping.

Direction: Addressed by Layer 3 in Section 6.

Destructive Mutability (The Time Problem)

Standard filesystems are mutable in place. When an agent overwrites a file, the previous version is lost. This creates a desynchronization between the agent’s “mind” (which can backtrack via context manipulation) and the “world” (which cannot easily rollback side-effects).

Manifestation: An agent that makes a mistake cannot undo filesystem changes without external versioning (git, snapshots). The cognitive ability to “reconsider” does not extend to effects.

Direction: Addressed by Layer 4 in Section 6.

Topological Rigidity (The Mesh Problem)

POSIX pipes imply local, linear composition. Scaling to distributed swarms requires abandoning the file metaphor for application-layer protocols (HTTP, gRPC), creating a semantic fracture between local and remote agents.

Manifestation: `cat file | ./quine "process" | ./quine "review"` works locally. Distributing these agents across machines requires replacing pipes with network protocols, losing the compositional simplicity.

Direction: Addressed by Layer 5 in Section 6.

Note on Scope

The first four limitations are architectural—addressable through OS-level extensions. This paper focuses on the POSIX mapping (Layer 1); the extensions are future work included to clarify what POSIX alone cannot provide.

Beyond POSIX: Extending the Foundation

The POSIX mapping presented in this paper represents **Layer 1** of a broader architectural vision. While process mapping provides robust lifecycle and resource management, internal agent state remains fragmented across files, descriptors, environment variables, and external logs.

These directions are included to clarify the limits of POSIX, not to claim that the present paper realizes a complete multi-layer architecture. My ongoing work extends Quine using Plan 9 principles (Pike et al. 1995). I outline four directions, each addressing an insufficiency exposed by the previous layer:

Layer 2 — Structure: Exposing Agent State

A synthetic filesystem exposes agent state through a unified namespace:

```
/agent/<pid>/tape      # Conversation history
/agent/<pid>/mission    # Current argv
/agent/<pid>/context    # Assembled prompt
/agent/<pid>/tools      # Available tool schemas
```

This makes the agent directly inspectable with standard Unix tools: `cat /agent/1234/tape` shows the conversation history, `tail -f /agent/1234/context` streams the evolving prompt. Debugging becomes exploration of a filesystem rather than log parsing.

Addresses: The Structure Problem (Section 5.2)—agent state becomes visible and queryable.

Layer 3 — Space: Scoped Namespaces

Namespace isolation and overlay-based private filesystems scope each agent’s operational world. An agent can be given a view that contains only the files relevant to its task:

```
# Agent sees only project directory, not full filesystem
./quine --mount /workspace:./project "Review this codebase"
```

Mutations become observable through the overlay: the agent can modify files without affecting the underlying filesystem, and all changes are recorded.

Addresses: The Space Problem (Section 5.2)—agents operate in constrained, task-appropriate views.

Layer 4 — Time: Snapshot-Based Rollback

Snapshot-based rollback enables independent restoration of cognitive state (context/history) and world state (filesystem). An agent that makes a mistake can:

1. Rollback the filesystem to a prior snapshot
2. Optionally rollback cognitive state to a prior checkpoint
3. Resume execution from the restored state

This allows execution to be revised without requiring cognitive and world state to move in lockstep. Branching becomes possible: explore multiple approaches from the same starting point.

Addresses: The Time Problem (Section 5.2)—effects become reversible.

Layer 5 — Mesh: Network Transparency

Network transparency via 9P (Pike et al. 1995) mounts remote agents as local directories:

```
# Remote agent appears as local filesystem
mount 9p://cluster-node/agent/5678 /remote-agent

# Pipe to remote agent using standard streams
```

```
cat data.txt | /remote-agent/stdin
cat /remote-agent/stdout
```

This preserves the file-based interface across machines, eliminating the semantic fracture between local and distributed composition.

Addresses: The Mesh Problem (Section 5.2)—remote agents compose like local ones.

Status

These layers are future work. Layer 2 has a partial prototype; Layers 3–5 are design-phase. The present paper focuses exclusively on Layer 1, which is complete and operational.

Conclusion

Quine shows that LLM agents can be architected as native POSIX processes, offering a different design point from application-layer frameworks. By mapping agents onto POSIX processes, Quine inherits well-understood process semantics for isolation, composition, and lifecycle control—mechanisms refined over fifty years.

The cost is accepting Unix’s assumptions: deterministic composition through pipes, text-stream interfaces, and shared-nothing isolation between processes. These constraints shape what agents can easily express and what requires additional mechanism.

This paper specifies **Layer 1** of a five-layer architecture. The POSIX mapping provides identity, lifecycle, and resource management; the Plan 9-inspired extensions outlined in Section 6 suggest one possible path beyond this foundation. I offer Quine as both a working implementation and an architectural basis for agent runtimes that inherit OS mechanisms rather than reimplementing them at the application layer. Whether OS-native runtimes offer advantages for large-scale agent systems remains an empirical question for future work.

Source code: <https://github.com/kehao95/quine>

CrewAI, Inc. 2024. *CrewAI: Framework for Orchestrating Role-Playing Autonomous AI Agents*. <https://github.com/crewAIInc/crewAI>.

IEEE. 2017. *Standard for Information Technology—Portable Operating System Interface (POSIX)*. IEEE Std 1003.1-2017. IEEE.

LangChain, Inc. 2023. *LangChain: Building Applications with LLMs Through Composability*. <https://github.com/langchain-ai/langchain>.

Li, Chen, Xiaoyu Liu, Xiang Meng, and Xin Zhao. 2026. “Architecting AgentOS: From Token-Level Context to Emergent System-Level Intelligence.” *arXiv Preprint arXiv:2602.20934*.

McIlroy, M. D. 1978. “Unix Time-Sharing System: Foreword.” *Bell System Technical Journal* 57 (6): 1899–904.

Mei, Kai, Zelong Li, Shuyuan Xu, Ruosong Ye, Yingqiang Ge, and Yongfeng Zhang. 2024. “AIOS: LLM Agent Operating System.” *arXiv Preprint arXiv:2403.16971*.

OpenAI. 2024. *Multi-Turn Retrieval Context Reasoning (MRCR) Benchmark*. <https://huggingface.co/datasets/openai/multi-turn-retrieval-context-reasoning>.

Pike, Rob, Dave Presotto, Sean Dorward, et al. 1995. “Plan 9 from Bell Labs.” *Computing Systems* 8 (3): 221–54.

Wallace, Eric, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. 2024. “The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions.” *arXiv Preprint arXiv:2404.13208*.

Wang, Xingyao, Boxuan Chen, Ziniu Adelt, et al. 2024. “OpenHands: An Open Platform for AI Software Developers as Generalist Agents.” *arXiv Preprint arXiv:2407.16741*.

Wu, Qingyun, Gagan Bansal, Jieyu Zhang, et al. 2023. “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework.” *arXiv Preprint arXiv:2308.08155*.

Yang, John, Carlos E. Jimenez, Alexander Wettig, Kilian Liber, Karthik Narasimhan, and Ofir Press. 2024. “SWE-Agent: Agent-Computer Interfaces Enable Automated Software Engineering.” *arXiv Preprint arXiv:2405.15793*.

Zhang, Chaoyun, He Huang, Chao Ni, et al. 2025. “UFO2: The Desktop AgentOS.” *arXiv Preprint arXiv:2504.14603*.